Informe Taller Hilos.

Integrantes: Lineth Paola Márquez Hernández y Jair Santiago Garzón Arias.

1. Consultas a sensores (`sensores.py`)

¿De qué va el código?

El programa simula la lectura de tres sensores independientes con distintos tiempos de respuesta: **Sensor A (1.0s)**, **Sensor B (2.0s)** y **Sensor C (1.5s)** usando `sleep()`. Se midió el tiempo total de ejecución usando `perf_counter()` en tres escenarios distintos, repitiendo cada prueba 3 veces para sacar la mediana.

Tabla de Resultados

Modo	Tiempo previsto	Mediana medida	Orden de finalización
Secuencial, con llamadas directas	4.500s	4.502s aprox.	A - B - C
Iniciar todos los hilos y después esperarlos	2.000s	2.002s aprox.	A - C - B
Ejecutar `join()` después de cada `start()`	4.500s	4.502s aprox.	A - B - C

Explicación: ¿Por qué cambian los tiempos según cómo se use `join()`?

La clave está en el momento en que bloqueamos el hilo principal:

1. **Iniciar todos y luego esperarlos (`start()` en bucle -> `join()` en bucle):** Al lanzar los tres hilos seguidos, las esperas (`sleep`) se solapan. Mientras el Sensor B espera sus 2s, los sensores A y C ya van avanzando y terminan antes 1.0s y 1.5s. El tiempo total lo marca únicamente el sensor más lento 2.0s.
2. **`join()` justo después de cada `start()`:** El método `join()` obliga al programa principal a quedarse quieto hasta que ese hilo específico termine. Si hacemos `hilo.start()` y de inmediato `hilo.join()`, el programa no crea el Sensor B hasta

que el A haya terminado por completo. En la práctica, esto convierte el código multihilo en una **ejecución secuencial** 1.0 + 2.0 + 1.5 = 4.5s.

Salidas obtenidas en consola.

```
========== RESULTADOS FINALES ==========
Secuencial: 4.501 segundos
Todos los hilos y después esperarlos: 2.001 segundos
Join después de cada start: 4.503 segundos
```

2. Dos cajas, una unidad (`inventario.py`)

¿De qué va el código?

Simula dos cajas (hilos) intentando vender al mismo tiempo la última unidad disponible de un producto (`inventario = 1`).

- **Versión sin `Lock`:** Usa una `Barrier(2)` para forzar que ambas cajas consulten el inventario al mismo tiempo antes de descontar. Como ambas ven que hay `1` unidad disponible, las dos aprueban la venta.
- **Versión con `Lock`:** Usa un candado (`candado`) que envuelve tanto la lectura como la resta del inventario, asegurando exclusión mutua.

Salidas obtenidas en consola.

```
========== VERSION SIN LOCK ==========
Caja 1 consulta inventario: 1
Caja 2 consulta inventario: 1
Caja 1 decision de venta: True
Caja 2 decision de venta: True
Caja 1: venta aprobada
Caja 2: venta aprobada

Resultados sin Lock:
Ventas aprobadas: 2
Inventario final: -1

========== VERSION CON LOCK ==========
Caja 1 consulta inventario: 1
Caja 1: venta aprobada
Caja 2 consulta inventario: 0
Caja 2: venta rechazada

Resultados con Lock:
Ventas aprobadas: 1
Inventario final: 0
Verificacion correcta: una sola venta aprobada
```

Explicación: ¿Por qué proteger solo el descuento sería insuficiente?

El problema en este ejercicio es una **condición de carrera (*race condition*)** de tipo *"verificar y luego actuar"*.

Si pusiéramos el `Lock` únicamente en la línea `inventario -= 1`, ambas cajas podrían entrar al `if inventario > 0:` al mismo tiempo, ver que hay `1` unidad, y tomar la decisión de vender. Luego pasarían por turnos al candado para restar: la primera dejaría el inventario en `0` y la segunda, como ya había tomado la decisión antes de entrar al candado, restaría igual y lo dejaría en `-1`. **Toda la operación (consultar + decidir + descontar) debe ser atómica dentro del mismo `Lock`.**

3. Lista de participantes en el chat TCP (`servidor_chat.py`)

¿De qué va el código?

El sistema implementa un chat grupal (broadcast) sobre TCP donde el servidor crea un hilo dedicado (atender_cliente) para cada usuario que se conecta. Cuando alguien escribe un mensaje normal, el servidor lo reenvía a todos los demás participantes conectados.

Aunque en este chat no existen mensajes privados, el comando /usuarios cumple dos funciones clave:

1. Orientación en la sala: Permite a cualquier usuario saber quiénes están conectados en ese momento antes de escribir un mensaje al grupo (saber a quién le va a llegar lo que escriba o confirmar si hay alguien en línea).
2. Prueba de concurrencia en memoria compartida: Sirve para consultar de forma segura el diccionario global usuarios = {} mientras múltiples hilos se conectan, envían mensajes o salen simultáneamente.

Salidas obtenidas en consola.

```
Escribe tu nombre:
juan
>
Bienvenido, juan. Comandos: /usuarios, /salir

[Sistema] santiago se ha conectado.

[Sistema] ana se ha conectado.

[ana] hola
buenas tardes
>
[santiago] holaaa
```

Respuestas a las preguntas de la guía

1. **¿Qué hilo recibe mensajes en el cliente mientras la terminal espera al usuario?**
   En cliente_chat.py, mientras el hilo principal (main) se queda bloqueado esperando a que el usuario escriba algo en input('> '), el hilo secundario receptor (que corre en segundo plano como daemon ejecutando la función recibir) sigue escuchando el socket de forma continua. Gracias a esto, Ana puede recibir el mensaje de Luis o los avisos del sistema en tiempo real sin tener que presionar Enter ni interrumpir lo que esté escribiendo.

2. **¿Por qué el envío de la lista ocurre fuera del candado del registro (registro_lock)?**
   El envío de datos por red (sendall) es una operación de entrada/salida (I/O) que puede tardar si la conexión de un cliente está lenta o congestionada. Si mantuviéramos bloqueado registro_lock mientras enviamos el mensaje por el socket, bloquearíamos a todos los demás hilos del servidor que necesiten conectarse, desconectarse o consultar el diccionario usuarios.
   Por eso, la estrategia correcta es:
   - Adquirir registro_lock solo por un microsegundo para copiar y ordenar las llaves (sorted(usuarios.keys())).
   - Liberar registro_lock inmediatamente.
   - Enviar la respuesta usando únicamente el candado individual de esa conexión (envio_lock), evitando así que se mezclen bytes si el servidor intenta escribirle dos cosas distintas a ese mismo cliente al mismo tiempo.